# Building Agentic AutoML

An educational project to build a useful agentic AutoML system step by step

**Project Report - Initial Design Edition**

| OBSERVE | DECIDE | ACT | EVALUATE | REMEMBER | RETURN |
|---|---|---|---|---|---|

Initial project specification - August 2026

# 1. Executive Summary

**Agentic AutoML** is a versioned educational project whose goal is to build, in public and step by step, an AutoML agent for tabular supervised learning. Given a dataset and a target column, the mature system should understand whether the task is binary classification, multiclass classification or regression; inspect the data; run controlled experiments; compare preprocessing and model strategies; optimize promising solutions; and return the best valid pipeline together with a complete decision trace.

**The project is intentionally simple at the beginning. Each version introduces one main capability and keeps previous components as stable as possible.**

# 2. Project Goal and Scope

- **Educational:** make agent architecture observable and understandable from the notebook itself.

- **Professional:** use sound validation, reproducibility, experiment tracking and explicit limitations.

- **Useful:** converge in a limited number of versions toward a system that can produce a strong tabular ML solution on Kaggle-style datasets.

- **Traceable:** never return only a score; preserve what was tried, what failed, what improved and why the final method was selected.

## Current boundaries

- The first versions are not intended to outperform mature AutoML libraries.

- The project initially targets tabular supervised learning only.

- LLMs and agent frameworks are deliberately excluded from early versions so the decision loop remains visible.

# 3. Educational Method

The project follows the same progressive construction method used in the previous Building AI Agent work: **one version, one concept, one architectural improvement**. A version must already run end to end, and a new abstraction is introduced only when a concrete limitation appears. Complexity is added because the previous version needs it, not because the final architecture is known in advance.

## Design principles

- Every notebook runs independently from top to bottom.

- Functions stay short and explicit; abstractions are introduced only when they solve a visible problem.

- The main agent function reads almost like pseudocode and coordinates the components instead of hiding their logic.

- Outputs expose state and experiment history rather than only polished summaries.

- Each new capability is demonstrated on a dataset or experiment chosen specifically to make its value visible.

# 4. Agent Architecture

The mature architecture is organized around a small recurring loop. Early versions implement only a subset of it; later versions progressively enrich each stage.

| | |
|---|---|
| **1. Observe** | Inspect dataset structure, target behavior and the evidence produced so far. |
| **2. Decide** | Choose the next useful experiment: metric, validation, preprocessing, model or optimization step. |
| **3. Act** | Execute one bounded experiment through small, explicit functions. |
| **4. Evaluate** | Measure performance consistently and detect invalid or failed experiments. |
| **5. Remember** | Store experiment configuration, score, duration, status and decision rationale. |
| **6. Return** | Select the best valid pipeline and explain the path that led to it. |

# 5. Version Roadmap

The roadmap is intentionally compact: eight versions are enough to move from a tiny baseline to a genuinely useful system without creating dozens of artificial milestones.

| Version | Name | What changes | Main lesson |
|---|---|---|---|
| **V01** | **Baseline Agent** | Detect the supervised task and train a fast LightGBM baseline. | Minimal end-to-end agent |
| **V02** | **Data Inspector** | Inspect target, types, missing values, cardinality and dataset size. | Observation before action |
| **V03** | **Preprocessing Agent** | Try a small set of preprocessing strategies and keep evidence of outcomes. | Data handling decisions |
| **V04** | **Model Selection** | Compare multiple model families with a consistent evaluation protocol. | Model choice by evidence |
| **V05** | **Smart Validation** | Choose stronger validation and metrics; introduce leakage awareness. | Reliable evaluation |
| **V06** | **Feature Engineering** | Test simple, justified feature transformations without hiding the logic. | Controlled feature search |
| **V07** | **Optimization** | Use Optuna only on promising candidates under an explicit budget. | Efficient tuning |
| **V08** | **Senior Agent** | Coordinate experiments, stop unproductive paths and return the best traceable pipeline. | Useful agentic AutoML system |

**Rule: a new version changes the smallest possible part of the previous one.**

# 6. Dataset Strategy

The project uses a single Kaggle Dataset containing multiple CSV files. Difficulty increases gradually and the same dataset can be reused in later versions to show whether a new capability creates a measurable improvement. The agent receives a dataset and a target; task type is inferred by the agent rather than read from a hidden answer key.

| File | Task | Level | Why it is included |
|---|---|---|---|
| **simple_binary.csv** | Binary classification | Very easy | Clean first baseline; mixed numeric/categorical features. |
| **simple_regression.csv** | Regression | Very easy | Mirrors V01 on a continuous target. |
| **adult_income.csv** | Binary classification | Medium | Missing values, mixed types, realistic categorical features. |
| **abalone.csv** | Regression | Medium | Small regression benchmark with numeric and categorical inputs. |
| **bank_marketing.csv** | Binary classification | Medium | Larger mixed-type classification and realistic imbalance. |
| **wine_quality.csv** | Regression / ambiguous | Medium | Numeric target that can provoke task-definition reasoning. |
| **student_dropout.csv** | Multiclass classification | Hard | Three classes, imbalance and richer feature space. |
| **bike_sharing.csv** | Regression | Hard | Temporal structure and intentional leakage candidates for later versions. |

The first two files are synthetic project datasets. The real benchmark collection is planned around well-known public tabular datasets, packaged as normalized CSV files in one Kaggle Dataset.

# 7. Notebook, Code and Output Style

- **Notebook structure:** Goal, Version N, Imports, Data, Components, Agent, Run, Results, Notes.

- **Markdown:** short explanations answering what changes, why it changes and what remains unchanged.

- **Code:** small functions, explicit names, minimal dependencies and no premature class hierarchy.

- **State:** simple Python dictionaries at first, evolving only when the project needs richer experiment history.

- **Results:** raw decision trace plus concise tables or plots when they add real information.

| Goal | Version N | Imports | Data | Components | Agent | Run | Results | Notes |
|---|---|---|---|---|---|---|---|---|

# 8. Experiment State and Decision Trace

A central design requirement is observability. Every experiment should eventually preserve at least: experiment identifier, task and metric, data strategy, model, hyperparameters, validation scheme, score, duration, status, failure reason when present, and the decision that followed. This history allows the final agent to answer not only *what won*, but *how it arrived there*.

| experiment_id | task | metric | preprocessing | model | params | validation | score | duration | status | decision |
|---|---|---|---|---|---|---|---|---|---|---|

# 9. Expected Final Capability

- Recognize regression, binary classification and multiclass classification.
- Inspect numeric and categorical features, missing values, cardinality, target behavior and dataset scale.
- Choose suitable metrics and validation strategies.
- Compare multiple preprocessing approaches and model families.
- Identify obvious leakage and invalid experimental setups in mature versions.
- Use Optuna only after narrowing the search to promising candidates.
- Return the best reproducible pipeline and a readable experiment history.

# 10. Current Status

This document describes the initial project design before V01 is implemented. The benchmark dataset collection and naming scheme have been defined, and the next development milestone is **V01 - Baseline Agent**. Experimental results are intentionally absent from this edition and will be added only when produced by executed notebooks.

## Next milestone

**Build V01 as the smallest complete system: load a CSV, receive the target, infer a simple task type, train a fast LightGBM baseline, evaluate it and return a transparent state.**

This report is a living project document. It should evolve only when executed notebook versions provide new evidence, decisions and results.

# Project Infographic